ЛАБОРАТОРНА РОБОТА №3

СИСТЕМНЕ ПРОГРАМУВАННЯ

ПАРАЛЕЛІЗМ ДАНИХ ТА ПАРАЛЕЛІЗМ ЗАДАЧ (Ч.2)

Заяць Діани Юріївни

Хід виконання роботи:

3.1. Створення програми для обробки масивів різних типів, розмірів і з різною складністю обчислень

Для виконання першого завдання було створено консольний проєкт lab3, у якому досліджується швидкодія послідовної та паралельної обробки масивів. У програмі використовуються масиви типів int і double, а також кілька розмірів масивів, наприклад 100000, 500000 і 1000000. Для кожного масиву виконуються обчислення за кількома формулами різної складності:

- x = x / 10
- x = x / π
- x = e^x / x^π
- x = e^(πx) / x^π

У програмі реалізовано окремі методи для:

- створення масиву цілих чисел;
- створення масиву дійсних чисел;
- послідовної обробки масиву;
- паралельної обробки масиву;
- вимірювання часу виконання за допомогою Stopwatch.

Для коректного порівняння часу виконання перед кожним тестом вихідний масив копіюється у дві окремі копії: одну для послідовного виконання, другу для паралельного. Це потрібно для того, щоб обидва режими працювали з

однаковими початковими даними. Таке рішення робить експеримент коректним і чесним.

Окремо в програмі передбачено допоміжну функцію безпечного перетворення результату в тип int, оскільки складні математичні обчислення можуть призводити до переповнення або появи некоректних значень. Це особливо важливо для формул з експонентою.

У методі Main() задаються розміри масивів і запускаються серії експериментів для int та double. Для кожного розміру викликаються окремі методи, які запускають усі формули та виводять результати у вигляді таблиці.

Методи BuildIntArray() і BuildDoubleArray() створюють та ініціалізують вихідні масиви. Значення добираються так, щоб уникнути ділення на нуль і нестабільної поведінки обчислень.

Метод послідовної обробки використовує звичайний цикл for, а метод паралельної обробки — Parallel.For(0, data.Length, i => ...). У паралельному варіанті кожен потік обробляє свій набір індексів масиву незалежно від інших. Це є прикладом паралелізму даних, коли одна й та сама операція застосовується до великої кількості незалежних елементів.

```
private const double Pi = Math.PI;

0 references
static void Main(string[] args)
{
    // Набір розмірів масивів для порівняння впливу обсягу даних на швидкодію.
    int[] sizes = { 100000, 500000, 1000000 };

    Console.WriteLine("Experiments: Parallel.For() vs sequential loop");
    Console.WriteLine(new string('-', 106));
    Console.WriteLine(
        "{0,-8} | {1,9} | {2,-24} | {3,10} | {4,12}",
        "Type",
        "Count",
        "Operation",
        "Seq., ms",
        "Par., ms");
    Console.WriteLine(new string('-', 106));

    foreach (int size in sizes)
    {
        // Для кожного розміру окремо тестуємо int і double.
        RunIntExperiments(size);
        RunDoubleExperiments(size);
    }

    Console.WriteLine(new string('-', 106));
    Console.ReadKey();
```

Рис. 3.1.1 — Код методу Main()

```
private static int[] BuildIntArray(int size)
{
    int[] array = new int[size];
    // Фіксоване зерно генератора робить експеримент відтворюваним.
    Random random = new Random(42 + size);
    for (int i = 0; i < array.Length; i++)
    {
        array[i] = random.Next(1, 10);
    }

    return array;
}

1 reference
private static double[] BuildDoubleArray(int size)
{
    double[] array = new double[size];
    // Фіксоване зерно генератора робить експеримент відтворюваним.
    Random random = new Random(84 + size);
    for (int i = 0; i < array.Length; i++)
    {
        array[i] = 1.0 + random.NextDouble() * 9.0;
    }

    return array;
}
```

Рис. 3.1.2 — Код методів створення масивів BuildIntArray() і BuildDoubleArray()

```
private static long MeasureSequential(int[] data, Func<int, int> operation)
{
    Stopwatch stopwatch = Stopwatch.StartNew();
    // Базовий (еталонний) варіант без паралелізму.
    for (int i = 0; i < data.Length; i++)
    {
        data[i] = operation(data[i]);
    }

    stopwatch.Stop();
    return stopwatch.ElapsedMilliseconds;
}

1 reference
private static long MeasureParallel(int[] data, Func<int, int> operation)
{
    Stopwatch stopwatch = Stopwatch.StartNew();
    // Розпаралелюємо обробку: кожен індекс і обчислюється незалежно.
    Parallel.For(0, data.Length, i => data[i] = operation(data[i]));
    stopwatch.Stop();
    return stopwatch.ElapsedMilliseconds;
}

1 reference
private static long MeasureSequential(double[] data, Func<double, double> operation)
{
    Stopwatch stopwatch = Stopwatch.StartNew();
    // Базовий (еталонний) варіант без паралелізму.
    for (int i = 0; i < data.Length; i++)
    {
        data[i] = operation(data[i]);
    }

    stopwatch.Stop();
    return stopwatch.ElapsedMilliseconds;
}

1 reference
private static long MeasureParallel(double[] data, Func<double, double> operation)
{
    Stopwatch stopwatch = Stopwatch.StartNew();
    // Розпаралелюємо обробку: кожен індекс і обчислюється незалежно.
    Parallel.For(0, data.Length, i => data[i] = operation(data[i]));
    stopwatch.Stop();
    return stopwatch.ElapsedMilliseconds;
```

Рис. 3.1.3 — Код послідовної обробки MeasureSequential(...) та паралельної обробки MeasureParallel(...)

```
private static void MeasureAndPrint(
    string typeName,
    int size,
    string operationName,
    int[] source,
    Func<int, int> operation)
{
    // Клонуємо масив, щоб послідовний і паралельний тести працювали з однаковими даними.
    int[] sequentialData = (int[])source.Clone();
    int[] parallelData = (int[])source.Clone();

    long sequentialTime = MeasureSequential(sequentialData, operation);
    long parallelTime = MeasureParallel(parallelData, operation);

    Console.WriteLine(
        "{0,-8} | {1,9} | {2,-24} | {3,10} | {4,12}",
        typeName,
        size,
        operationName,
        sequentialTime,
        parallelTime);
}

4 references
private static void MeasureAndPrint(
    string typeName,
    int size,
    string operationName,
    double[] source,
    Func<double, double> operation)
{
    // Клонуємо масив, щоб послідовний і паралельний тести працювали з однаковими даними.
    double[] sequentialData = (double[])source.Clone();
    double[] parallelData = (double[])source.Clone();

    long sequentialTime = MeasureSequential(sequentialData, operation);
    long parallelTime = MeasureParallel(parallelData, operation);

    Console.WriteLine(
        "{0,-8} | {1,9} | {2,-24} | {3,10} | {4,12}",
        typeName,
        size,
        operationName,
        sequentialTime,
        parallelTime);
```

Рис. 3.1.5 — Код методу MeasureAndPrint(...)

```
Experiments: Parallel.For() vs sequential loop
-----------------------------------------------------------------------------------------------------------
Type     |     Count | Operation                |   Seq., ms |     Par., ms
-----------------------------------------------------------------------------------------------------------
int      |    100000 | x = x / 10               |          0 |           20
int      |    100000 | x = x / pi               |          0 |            1
int      |    100000 | x = e^x / x^pi           |          6 |           21
int      |    100000 | x = e^(pi*x)/x^pi        |          7 |            2
double   |    100000 | x = x / 10               |          0 |            0
double   |    100000 | x = x / pi               |          0 |            0
double   |    100000 | x = e^x / x^pi           |         31 |            1
double   |    100000 | x = e^(pi*x)/x^pi        |          4 |            1
int      |    500000 | x = x / 10               |          1 |            1
int      |    500000 | x = x / pi               |          1 |            5
int      |    500000 | x = e^x / x^pi           |         28 |            6
int      |    500000 | x = e^(pi*x)/x^pi        |         29 |            6
double   |    500000 | x = x / 10               |          1 |            2
double   |    500000 | x = x / pi               |          2 |            1
double   |    500000 | x = e^x / x^pi           |         20 |            8
double   |    500000 | x = e^(pi*x)/x^pi        |         20 |            2
int      |   1000000 | x = x / 10               |          5 |            1
int      |   1000000 | x = x / pi               |         24 |            1
int      |   1000000 | x = e^x / x^pi           |         51 |           65
int      |   1000000 | x = e^(pi*x)/x^pi        |         64 |           12
double   |   1000000 | x = x / 10               |          5 |            2
double   |   1000000 | x = x / pi               |          3 |            1
double   |   1000000 | x = e^x / x^pi           |         44 |            5
double   |   1000000 | x = e^(pi*x)/x^pi        |         45 |            4
-----------------------------------------------------------------------------------------------------------
```

Рис. 3.1.6 — Результат виконання програми

За результатами експериментів можна зробити висновок, що паралельна обробка даних є доцільною насамперед для великих масивів і складних обчислень. Для простих операцій або невеликих обсягів даних виграш може бути незначним або навіть відсутнім через накладні витрати на керування потоками. Для складних математичних виразів паралельне виконання, як правило, виявляється ефективнішим.

3.2. Модифікація програми для виходу з паралельного циклу за умови входження значення елемента в окол заданого числа

Для виконання другого завдання було створено окремий проєкт ParallelForBreak. У ньому реалізовано пошук елемента, значення якого потрапляє в окол заданого числа. Число і допустиме відхилення задаються константами, наприклад:

- цільове число TargetValue;
- допустиме відхилення Tolerance.

Під час виконання циклу Parallel.For() перевіряється умова належності значення елемента до цього околу. Якщо умова виконується, зберігається індекс знайденого елемента, а цикл зупиняється через керування станом циклу.

У реалізації для захисту від ситуації, коли кілька потоків майже одночасно знайдуть підходяще значення, використовується Interlocked.CompareExchange. Це дозволяє безпечно записати індекс лише один раз. Для зупинки обчислень використовується state.Stop(), що є логічним рішенням у задачі пошуку, де достатньо знайти будь-який відповідний елемент.

У методі Main() створюється масив вхідних даних і запускається Parallel.For() з двома параметрами в тілі циклу: індексом та об’єктом стану циклу ParallelLoopState.

У тілі циклу:

1. зчитується значення елемента;

2. виконується обчислення або перевірка;
3. перевіряється входження в заданий окол числа;
4. у разі успіху індекс зберігається атомарно;
5. викликається зупинка паралельного циклу.

Після завершення циклу аналізується об’єкт ParallelLoopResult, за допомогою якого можна визначити, чи був цикл завершений достроково.

```
private const double TargetValue = 123.45;
private const double Tolerance = 0.02;

0 references
private static void Main(string[] args)
{
    // Формуємо масив для демонстрації пошуку з достроковою зупинкою циклу.
    double[] data = BuildData(1000000);

    // foundIndex = -1 означає, що підходящий елемент ще не знайдено.
    int foundIndex = -1;
    double foundValue = 0.0;

    ParallelLoopResult result = Parallel.For(0, data.Length, (i, state) =>
    {
        double x = data[i];
        // Імітуємо обчислювальне навантаження для кожного елемента.
        data[i] = Math.Exp(x) / Math.Pow(x, Math.PI);

        // Якщо значення потрапляє в окіл цілі — ініціюємо дострокову зупинку.
        if (Math.Abs(x - TargetValue) <= Tolerance)
        {
            // Фіксуємо лише перший знайдений індекс у потокобезпечний спосіб.
            if (Interlocked.CompareExchange(ref foundIndex, i, -1) == -1)
            {
                foundValue = x;
            }

            // Stop() зупиняє запуск нових ітерацій якнайшвидше.
            state.Stop();
        }
    });

    Console.WriteLine("Searching for an element within a target neighborhood (Parallel.For + Stop):");
    Console.WriteLine("Target = {0}, Tolerance = {1}", TargetValue, Tolerance);
    // Якщо цикл зупинили достроково, IsCompleted буде false.
    Console.WriteLine("IsCompleted = {0}", result.IsCompleted);
    Console.WriteLine(
        foundIndex >= 0
            ? string.Format("Found: index={0}, value={1:F5}", foundIndex, foundValue)
            : "Element not found.");
    Console.ReadKey();
}

1 reference
private static double[] BuildData(int size)
{
    double[] array = new double[size];
    Random random = new Random(123);
    for (int i = 0; i < array.Length; i++)
    {
        array[i] = 1.0 + random.NextDouble() * 200.0;
    }

    // Гарантуємо, що хоча б один елемент потрапить в заданий окіл.
    array[size / 2] = TargetValue + Tolerance / 2.0;
    return array;
}
```

Рис. 3.2.1 — Код програми

```
Searching for an element within a target neighborhood (Parallel.For + Stop):
Target = 123,45, Tolerance = 0,02
IsCompleted = False
Found: index=5885, value=123,45994
```

Рис. 3.2.2 — Результат виконання програми

У ході виконання другого завдання було реалізовано дострокове завершення паралельного циклу. Це показало, що паралельні конструкції в C# дозволяють не лише розподіляти обчислення між потоками, а й ефективно керувати логікою завершення обробки. Такий підхід особливо корисний у задачах пошуку, де після знаходження потрібного результату продовження обчислень уже не має сенсу.

3.3. Повторення прикладів розпаралелювання за допомогою Parallel.ForEach()

Для виконання третього завдання було створено окремий проєкт ParallelForEachExamples, який демонструє використання Parallel.ForEach() для паралельної обробки масиву. У цьому проєкті реалізовано два основні приклади:

1. порівняння часу послідовної і паралельної обробки;
2. достроковий вихід з циклу при виконанні певної умови.

У прикладі вимірювання часу спочатку створюється масив даних, потім його копії для послідовної та паралельної обробки. Послідовний варіант виконується через звичайний цикл, а паралельний — через Parallel.ForEach() з доступом до значення, стану циклу та індексу елемента. Такий варіант зручний тим, що дозволяє змінювати масив за індексом прямо всередині паралельного циклу.

У прикладі дострокового завершення один із елементів масиву робиться від'ємним. Під час обробки, якщо виявляється від'ємне значення, викликається state.Break(). Після завершення аналізується ParallelLoopResult, і виводиться інформація про те, що цикл не завершився повністю. Це відповідає вимозі методички повторити приклади вже через ForEach().

У цьому проєкті використовується перевантаження Parallel.ForEach(...), яке надає:

- поточне значення елемента;
- об'єкт ParallelLoopState;
- індекс елемента.

Це зручно, бо дозволяє одночасно виконувати обчислення, змінювати масив і за потреби переривати обробку. На відміну від звичайного foreach, тут цикл справді виконується в кількох потоках.

```
// Дані для прикладу з достроковим виходом через Break().
private static double[] breakData;

0 references
private static void Main(string[] args)
{
    // 1) Порівняння часу послідовної та паралельної обробки через ForEach.
    RunTimingExample();
    Console.WriteLine();
    // 2) Демонстрація дострокового завершення ForEach при знаходженні умови.
    RunBreakExample();
    Console.ReadKey();
}

1 reference
private static void RunTimingExample()
{
    double[] data = BuildData(800000);
    // Клонуємо масив для коректного порівняння двох підходів.
    double[] seq = (double[])data.Clone();
    double[] par = (double[])data.Clone();

    Stopwatch swSeq = Stopwatch.StartNew();
    for (int i = 0; i < seq.Length; i++)
    {
        seq[i] = Math.Exp(Math.PI * seq[i]) / Math.Pow(seq[i], Math.PI);
    }

    swSeq.Stop();

    Stopwatch swPar = Stopwatch.StartNew();
    // ForEach із доступом до index, щоб змінювати елементи вихідного масиву.
    Parallel.ForEach(par, (value, state, index) =>
    {
        par[index] = Math.Exp(Math.PI * value) / Math.Pow(value, Math.PI);
    });
    swPar.Stop();

    Console.WriteLine("ForEach example (complex calculation):");
    Console.WriteLine("Sequential: {0} ms", swSeq.ElapsedMilliseconds);
    Console.WriteLine("Parallel: {0} ms", swPar.ElapsedMilliseconds);
}

1 reference
private static void RunBreakExample()
{
    breakData = new double[200000];
    // Ініціалізуємо масив невід'ємними значеннями.
    for (int i = 0; i < breakData.Length; i++)
    {
        breakData[i] = i;
    }

    // Додаємо від'ємний елемент, який має викликати Break().
    breakData[100000] = -10;

    ParallelLoopResult loopResult = Parallel.ForEach(breakData, MyTransformWithBreak);

    Console.WriteLine("ForEach break example:");
```

```
    Console.WriteLine("ForEach break example:");
    if (!loopResult.IsCompleted)
    {
        // LowestBreakIteration показує найменший індекс, на якому спрацював Break().
        Console.WriteLine(
            "Parallel.ForEach was aborted with negative value on iteration {0}",
            loopResult.LowestBreakIteration);
    }
    else
    {
        Console.WriteLine("Parallel.ForEach completed all iterations.");
    }
}

1 reference
private static void MyTransformWithBreak(double value, ParallelLoopState state, long index)
{
    // Як тільки знайдено від'ємне значення — перериваємо подальші ітерації.
    if (value < 0)
    {
        state.Break();
    }
}

1 reference
private static double[] BuildData(int size)
{
    double[] array = new double[size];
    Random random = new Random(777);
    for (int i = 0; i < array.Length; i++)
    {
        array[i] = 1.0 + random.NextDouble() * 9.0;
    }

    return array;
}
```

Рис. 3.3.1 – Код програми

```
ForEach example (complex calculation):
Sequential: 31 ms
Parallel: 104 ms

ForEach break example:
Parallel.ForEach was aborted with negative value on iteration 100000
```

Рис. 3.3.2 — Результат виконання програми

У результаті виконання третього завдання було досліджено використання Parallel.ForEach() для паралельної обробки колекцій. Було встановлено, що цей метод зручний для роботи з масивами та іншими наборами даних, коли немає потреби явно керувати межами індексів циклу. Також було підтверджено, що Parallel.ForEach() підтримує механізми дострокового завершення циклу.

3.4 Модифікація програми, щоб тіло паралельного циклу задавалося лямбда-виразом

Для виконання четвертого завдання було створено окремий проєкт ParallelForEachLambda, у якому тіло паралельного циклу задається лямбда-виразом. Це відповідає вимозі методички реалізувати більш гнучкий варіант побудови паралельного циклу.

У програмі створюється змінна типу Action<double[], Func<double, double>>, яка зберігає логіку запуску паралельної обробки. Усередині цієї змінної використовується Parallel.ForEach(), а саму операцію обробки елемента передано окремо як функцію Func<double, double>. Таким чином, тіло циклу визначається не окремим іменованим методом, а безпосередньо лямбда-виразом.

Такий підхід робить код більш компактним, гнучким і придатним до повторного використання. При необхідності можна легко підставити іншу формулу, не змінюючи структуру циклу.

У коді створюється делегат, який приймає:

- масив даних;
- функцію перетворення одного елемента.

Далі всередині лямбда-виразу викликається Parallel.ForEach(), і для кожного елемента масиву застосовується передана функція. Це є прикладом поєднання паралельного програмування з функціональним стилем у C#.

```
0 references
private static void Main(string[] args)
{
    // Невеликий масив зручний для наочного виводу результатів у консоль.
    double[] data = BuildData(25);

    // Тіло паралельного циклу задаємо лямбда-виразом через Action + Func.
    Action<double[], Func<double, double>> runParallel = (array, transform) =>
        Parallel.ForEach(array, (value, state, index) => array[index] = transform(value));

    // Окрема лямбда-функція з формулою перетворення елемента.
    Func<double, double> operation = x => Math.Exp(Math.PI * x) / Math.Pow(x, Math.PI);

    // Запускаємо паралельну обробку масиву з переданою формулою.
    runParallel(data, operation);

    Console.WriteLine("Parallel.ForEach with lambda loop body:");
    for (int i = 0; i < data.Length; i++)
    {
        Console.WriteLine("data[{0}] = {1:F6}", i, data[i]);
    }

    Console.ReadKey();
}

1 reference
private static double[] BuildData(int size)
{
    double[] array = new double[size];
    // Фіксоване зерно -> однакові вхідні дані при кожному запуску.
    Random random = new Random(101);
    for (int i = 0; i < array.Length; i++)
    {
        array[i] = 1.0 + random.NextDouble() * 4.0;
    }

    return array;
}
```

Рис. 3.4.1 — Код програми

```
C:\Users\Lenovo Legion\OneD

Parallel.ForEach with lambda loop body:
data[0] = 364,994010
data[1] = 265,837834
data[2] = 135,154610
data[3] = 30,005893
data[4] = 6276,867141
data[5] = 7353,873981
data[6] = 36,483851
data[7] = 29,584142
data[8] = 313,799702
data[9] = 455,496908
data[10] = 2344,136308
data[11] = 96,534057
data[12] = 25,353296
data[13] = 23,271796
data[14] = 1906,831299
data[15] = 28,986759
data[16] = 95,966164
data[17] = 1067,253919
data[18] = 191,347328
data[19] = 208,606770
data[20] = 55,922839
data[21] = 1186,053523
data[22] = 25,109342
data[23] = 94,574672
data[24] = 919,361361
```

Рис. 3.4.4 — Результат виконання програми

У четвертому завданні було реалізовано паралельний цикл, тіло якого задається лямбда-виразом. Це дозволило зробити код коротшим, гнучкішим і більш універсальним. Використання лямбда-виразів разом із делегатами Action і Func спрощує передачу обчислювальної логіки в паралельні методи та підвищує зручність повторного використання коду.

Висновок:

У ході виконання лабораторної роботи було досліджено засоби паралельного програмування в C# на основі Parallel.For() та Parallel.ForEach(). Було реалізовано обробку масивів різних типів і розмірів, проведено порівняння часу послідовного та паралельного виконання, реалізовано достроковий вихід

із паралельного циклу за заданою умовою, а також створено варіант програми, в якому тіло циклу задається лямбда-виразом.

У результаті встановлено, що паралельна обробка є ефективною насамперед для великих обсягів даних і складних математичних обчислень. Для простих операцій прискорення може бути незначним через накладні витрати на організацію паралельного виконання. Також було з’ясовано, що механізми Break(), Stop() і ParallelLoopResult дозволяють ефективно керувати виконанням паралельних циклів. Лабораторна робота дозволила на практиці засвоїти базові принципи паралелізму даних і задач у середовищі C#.